Continuity: Password-Recoverable Authority over Threshold Signing Accounts

John Edward Charlton
x1e1 Labs
engineering@x1e1labs.com

Abstract. Recoverable signing accounts should ensure that provider-held state alone forms no signing quorum and that recovery requires no pre-provisioned user-held recovery material beyond a password. Continuity is a Client–Server orchestration protocol for password-recoverable authority over threshold signing accounts. Through an augmented password-authenticated key exchange, the Client derives a recovery key for a Client-side key-wrapping hierarchy, while a two-party threshold signature scheme produces signatures from a Client share recovered through that key hierarchy and a Server share. The protocol defines a hierarchical account model, operation and wrapped-artifact context binding, and authentication, signing, and lifecycle operations.

1 Introduction

Signing authority needs to survive device loss, but the recovery mechanism determines who can complete signing. Provider-operated signing keeps signing available after device loss through provider infrastructure, but can give that infrastructure a signing quorum it can satisfy without user-side signing participation. User-held key material keeps signing outside of provider control, but can tie recovery to pre-provisioned recovery material beyond a password.

Continuity targets the middle ground: password-recoverable authority over threshold signing accounts in which provider-held state alone forms no signing quorum and recovery requires no pre-provisioned user-held recovery material beyond a password.

2 Related Work

Turnkey centralizes key-management and signing workflows around hardware-isolated execution, while DFNS Cloud uses multi-party computation with threshold signature schemes across DFNS-operated signers to produce signatures without key reconstruction [1, 2]. In both models, all signing-key material cryptographically required to produce a signature resides within provider-operated infrastructure, and no user-held key share participates in the signature computation.

BIP 32 derives a key hierarchy from a seed, while BIP 39 derives a seed from a mnemonic [3, 4]. A user-held mnemonic makes recovery depend on the mnemonic rather than a password alone.

The Password Protected Universal Thresholdizer adds password authentication to threshold computation and signing, although an attacker who compromises enough servers can bypass the password and sign without the user [5]. Password-Protected Threshold Signatures instead requires the user to recover signing state with a password before signing jointly with a preset minimum number of servers [6]; neither work specifies an account lifecycle covering password change, adding signing suites, or replacing a suite's shares without changing its public key.

3 Foundations

Continuity assumes the following components:

aPAKE: An augmented password-authenticated key exchange (aPAKE) integration with registration and mutual authentication in which registration state and each authenticated exchange cryptographically bind an application-defined Client identity and the intended Server, with each authenticated exchange's cryptographic binding also covering the expected protocol identifier, version, and profile identifier; registration runs over a channel providing confidentiality, integrity, and independent Server authentication and yields a Client-only pseudorandom wrapping key recovery_key; authentication re-derives the recovery_key for that registration and derives a fresh transport_key cryptographically independent of recovery_key; persisted per-registration state alone does not permit offline verification of password guesses.

MPC-TSS: A threshold signature scheme (TSS) implemented through multi-party computation (MPC), supporting distributed key generation

(DKG), public-key-preserving resharing, and interactive threshold signing that leaves long-lived signing-share state unchanged, with unforgeability under its stated participant-corruption model and without full private-key reconstruction.

Canonical encoding: A canonical encoding that is deterministic and injective, with decoders that reject non-canonical encodings.

AEAD: An authenticated encryption with associated data (AEAD) scheme providing confidentiality and integrity, with either nonce-misuse resistance or a requirement that no nonce be reused with the same AEAD key.

CSPRNG: A cryptographically secure pseudorandom number generator (CSPRNG) providing fresh randomness.

4 Protocol

4.1 System Model

Continuity operates in a two-party Client–Server setting in which the user is the principal and the Client and Server are the user-side and provider-side protocol roles, respectively.

A user account u governs zero or more signing accounts. A signing account a is governed by one user account and governs one or more suites.

Authority is the standing to initiate, authorize, and manage account operations.

A suite σ is a signing account's threshold signing configuration requiring both Client and Server participation, with public verification key $pk_{a,\sigma}$, Client-recoverable signing share $x^{\mathsf{C}}_{a,\sigma}$, and Server-held signing share $x^{\mathsf{S}}_{a,\sigma}$. Each signing share contains its participant's secret key share and persistent scheme metadata required for signing and public-key-preserving resharing.

4.2 Context Binding

Continuity uses the context ctx:

$$\mathsf{ctx} = \{\mathsf{protocol},\ \mathsf{version},\ \mathsf{profileId},\ \mathsf{bindingCtx}\},$$

where protocol identifies the protocol and has the value continuity, version identifies the protocol version and has the value 0, profileId identifies the selected foundational components and their parameters, and bindingCtx contains the operation or artifact class in class and the other protocol-defined fields applicable to the bound item. When present in bindingCtx, userAccountId identifies u, signingAccountId identifies a, suiteId identifies σ, suiteIds contains a duplicate-free list of suiteId values, signPayload contains the exact application message to be signed, requestId identifies an operation instance, including all subprotocol executions and messages exchanged as part of it, and wrapEpoch identifies a wrapped artifact's generation, which starts at 0 when the artifact is first created and increments by one whenever it is replaced.

ctx is validated against its expected operation or artifact and canonically encoded before use; validation rejects malformed contexts, unexpected or unsupported field values, and identifier values required to be fresh but previously introduced in the same field within Continuity.

Each operation request includes ctx with a bindingCtx that includes the operation class, a fresh requestId, and, as applicable, userAccountId, signingAccountId, suiteId, suiteIds, and signPayload. Each wrapped artifact includes the ciphertext, any profile-required public salts and nonces, and a ctx whose encoding is used as additional authenticated data (AAD) and whose bindingCtx includes the artifact class, wrapEpoch, and, as applicable, userAccountId, signingAccountId, and suiteId.

4.3 Key Hierarchy

Continuity uses separate wrapping hierarchies for Client- and Server-protected material. The aPAKE derives recovery_key as the root of the Client-protected hierarchy, while the Server provides a wrapping key wrap_key as the root of the Server-protected hierarchy and manages the key's lifecycle outside Continuity. Each wrapping key has the security strength required by the selected profile, and all non-root wrapping keys are independently sampled when created.

On the Client-protected side, recovery_key wraps the user-account key-encryption key (KEK) K^{C}_u, which wraps each signing account's Client-side signing-account KEK K^{C}_a, and each K^{C}_a wraps per-suite $x^{\mathsf{C}}_{a,\sigma}$ values.

On the Server-protected side, wrap_key wraps each signing account's Server-side signing-account KEK K^{S}_a, and each K^{S}_a wraps per-suite $x^{\mathsf{S}}_{a,\sigma}$ values.

For a wrapping key k and value v, $k{:}v$ denotes v wrapped under k.

4.4 Authentication

Authentication establishes user accounts and establishes or terminates authenticated sessions. Except for Signup and Signin, the Server processes a Client-initiated operation only while the initiating authenticated session remains valid and only within that session's user-account hierarchy.

4.4.1 Signup

Signup establishes the user account. The Client submits a Signup request with class = signup, bound to a fresh userAccountId and requestId.

Using the canonical encoding of the Signup request's userAccountId as the Client identity, the Client and Server complete aPAKE registration for the password over the independently authenticated registration channel, yielding aPAKE registration state and recovery_key. The Client samples K_u^{C} and wraps it as recovery_key:K_u^{C} with class = userAccountKek, bound to userAccountId and wrapEpoch. The Client provides the aPAKE registration state and recovery_key:K_u^{C} to the Server, and the Server atomically persists them as user-account state if userAccountId is still unassigned.

4.4.2 Signin

Signin establishes an authenticated session. The Client submits a Signin request with class = signin, bound to userAccountId and requestId.

Using the canonical encoding of the Signin request's userAccountId as the Client identity and the current aPAKE registration state for that userAccountId, the Client and Server complete the aPAKE exchange and derive transport_key, while the Client re-derives recovery_key. Successful completion of the aPAKE exchange establishes an authenticated session for that userAccountId only if the registration state used is still current; every subsequent Continuity Client–Server message in that session is protected by an application-selected transport instantiated from transport_key that provides confidentiality, integrity, and replay protection. The Server responds with the user account's recovery_key:K_u^{C}, from which the Client recovers K_u^{C}.

4.4.3 Signout

Signout terminates an authenticated session. The Client submits a Signout request with class = signout, bound to userAccountId and requestId.

The Client and Server complete Signout by invalidating the corresponding authenticated-session state.

4.5 Lifecycle Operations

Lifecycle operations establish signing accounts, provision suites, and update persistent user-account, signing-account, and suite state. These operations commit atomically only if the state on which they depend is unchanged; a committed replacement preserves the replaced artifact's class and applicable account and suite identifiers, and the Server retires superseded persistent state.

4.5.1 Provisioning

Provisioning establishes a signing account with its initial suite or provisions an additional suite under an existing signing account. The Client submits a Provisioning request with class = provision, bound to userAccountId, signingAccountId, suiteId, and requestId. For a new signing account, signingAccountId is fresh; in either case, suiteId is unused within the target signing account.

For a new signing account, the Client samples K_a^{C}, and the Server samples K_a^{S}. The Client wraps K_a^{C} as K_u^{C}:K_a^{C} with class = clientSigningAccountKek, bound to userAccountId, signingAccountId, and wrapEpoch. The Server wraps K_a^{S} as wrap_key:K_a^{S} with class = serverSigningAccountKek, bound to userAccountId, signingAccountId, and wrapEpoch.

For an existing signing account, the Server recovers K_a^{S} from wrap_key:K_a^{S} and responds with K_u^{C}:K_a^{C}; the Client then recovers K_a^{C} from K_u^{C}:K_a^{C}.

The Client and Server run MPC-TSS DKG, yielding $pk_{a,\sigma}$, $x_{a,\sigma}^{\mathsf{C}}$ to the Client, and $x_{a,\sigma}^{\mathsf{S}}$ to the Server. The Client wraps $x_{a,\sigma}^{\mathsf{C}}$ as K_a^{C}:$x_{a,\sigma}^{\mathsf{C}}$ with class = clientShare, bound to userAccountId, signingAccountId, suiteId, and wrapEpoch. The Server wraps $x_{a,\sigma}^{\mathsf{S}}$ as K_a^{S}:$x_{a,\sigma}^{\mathsf{S}}$ with class = serverShare, bound to userAccountId, signingAccountId, suiteId, and wrapEpoch.

The Client provides K_a^{C}:$x_{a,\sigma}^{\mathsf{C}}$ to the Server in both cases. For a new signing account, the Client also provides K_u^{C}:K_a^{C}. The Server persists the new or updated signing-account state, including $pk_{a,\sigma}$, K_a^{C}:$x_{a,\sigma}^{\mathsf{C}}$, K_a^{S}:$x_{a,\sigma}^{\mathsf{S}}$, and any newly wrapped signing-account KEKs.

4.5.2 Password Change

Password Change updates the user account's password binding without changing K_u^{C}. The Client submits a Password Change request with class = passwordChange, bound to userAccountId and requestId.

Using the canonical encoding of the Password Change request's userAccountId as the Client identity, the Client and Server perform a new aPAKE registration for the replacement password, yielding replacement aPAKE registration state and recovery_key′, where a prime denotes a replacement key or value. The Client rewraps K_u^{C} as recovery_key′:K_u^{C}. The Client provides the replacement aPAKE registration state and recovery_key′:K_u^{C} to the Server, which persists them as the current aPAKE registration state and recovery_key:K_u^{C}, respectively, while keeping the initiating authenticated session valid under its existing transport_key with recovery_key′ as its current recovery_key and invalidating every other authenticated session for the user account.

4.5.3 Non-Root KEK Rotation

Non-Root KEK Rotation replaces a non-root wrapping key and rewraps the artifacts it protects.

For User-Account KEK Rotation, the Client submits a request with class = userAccountKekRotation, bound to userAccountId and requestId. The Server responds with $K_u^{\mathsf{C}}{:}K_a^{\mathsf{C}}$ for every signing account governed by the user account. The Client recovers each K_a^{C} from the corresponding $K_u^{\mathsf{C}}{:}K_a^{\mathsf{C}}$, samples $(K_u^{\mathsf{C}})'$, wraps it as recovery_key:$(K_u^{\mathsf{C}})'$, and rewraps each recovered K_a^{C} as $(K_u^{\mathsf{C}})'{:}K_a^{\mathsf{C}}$. The Client provides recovery_key:$(K_u^{\mathsf{C}})'$ and the $(K_u^{\mathsf{C}})'{:}K_a^{\mathsf{C}}$ values to the Server, which persists them as recovery_key:K_u^{C} and the current $K_u^{\mathsf{C}}{:}K_a^{\mathsf{C}}$ values, respectively, while keeping the initiating authenticated session valid under its existing transport_key with $(K_u^{\mathsf{C}})'$ as its current K_u^{C} and invalidating every other authenticated session for the user account.

For Client-Side Signing-Account KEK Rotation, the Client submits a request bound to userAccountId, signingAccountId, suiteIds, and requestId, with class = clientSigningAccountKekRotation, where suiteIds contains all current suiteId values for the signing account. The Server responds with $K_u^{\mathsf{C}}{:}K_a^{\mathsf{C}}$ and, for each suiteId value in suiteIds, the corresponding $K_a^{\mathsf{C}}{:}x_{a,\sigma}^{\mathsf{C}}$. The Client recovers K_a^{C} from $K_u^{\mathsf{C}}{:}K_a^{\mathsf{C}}$ and each $x_{a,\sigma}^{\mathsf{C}}$ from the corresponding $K_a^{\mathsf{C}}{:}x_{a,\sigma}^{\mathsf{C}}$. The Client samples $(K_a^{\mathsf{C}})'$, wraps it as $K_u^{\mathsf{C}}{:}(K_a^{\mathsf{C}})'$, and rewraps each recovered $x_{a,\sigma}^{\mathsf{C}}$ as $(K_a^{\mathsf{C}})'{:}x_{a,\sigma}^{\mathsf{C}}$. The Client provides $K_u^{\mathsf{C}}{:}(K_a^{\mathsf{C}})'$ and the $(K_a^{\mathsf{C}})'{:}x_{a,\sigma}^{\mathsf{C}}$ values to the Server, which persists them as $K_u^{\mathsf{C}}{:}K_a^{\mathsf{C}}$ and the current $K_a^{\mathsf{C}}{:}x_{a,\sigma}^{\mathsf{C}}$ values, respectively.

For Server-Side Signing-Account KEK Rotation, no request is required. For a selected signing account, the Server recovers K_a^{S} from wrap_key:K_a^{S} and each current $x_{a,\sigma}^{\mathsf{S}}$ from its $K_a^{\mathsf{S}}{:}x_{a,\sigma}^{\mathsf{S}}$. The Server samples $(K_a^{\mathsf{S}})'$, wraps it as wrap_key:$(K_a^{\mathsf{S}})'$, and rewraps each recovered $x_{a,\sigma}^{\mathsf{S}}$ as $(K_a^{\mathsf{S}})'{:}x_{a,\sigma}^{\mathsf{S}}$. The Server persists wrap_key:$(K_a^{\mathsf{S}})'$ and the $(K_a^{\mathsf{S}})'{:}x_{a,\sigma}^{\mathsf{S}}$ values as wrap_key:K_a^{S} and the current $K_a^{\mathsf{S}}{:}x_{a,\sigma}^{\mathsf{S}}$ values, respectively.

4.5.4 Share Refresh

Share Refresh replaces a suite's threshold shares while preserving its public key. The Client submits a Share Refresh request with class = shareRefresh, bound to userAccountId, signingAccountId, suiteId, and requestId.

The Server recovers K_a^{S} from wrap_key:K_a^{S} and $x_{a,\sigma}^{\mathsf{S}}$ from $K_a^{\mathsf{S}}{:}x_{a,\sigma}^{\mathsf{S}}$ and responds with $K_u^{\mathsf{C}}{:}K_a^{\mathsf{C}}$ and $K_a^{\mathsf{C}}{:}x_{a,\sigma}^{\mathsf{C}}$; the Client then recovers K_a^{C} from $K_u^{\mathsf{C}}{:}K_a^{\mathsf{C}}$ and $x_{a,\sigma}^{\mathsf{C}}$ from $K_a^{\mathsf{C}}{:}x_{a,\sigma}^{\mathsf{C}}$.

The Client and Server run MPC-TSS public-key-preserving resharing using their respective current shares, yielding $(x_{a,\sigma}^{\mathsf{C}})'$ to the Client and $(x_{a,\sigma}^{\mathsf{S}})'$ to the Server.

The Client wraps $(x_{a,\sigma}^{\mathsf{C}})'$ as $K_a^{\mathsf{C}}{:}(x_{a,\sigma}^{\mathsf{C}})'$, and the Server wraps $(x_{a,\sigma}^{\mathsf{S}})'$ as $K_a^{\mathsf{S}}{:}(x_{a,\sigma}^{\mathsf{S}})'$. The Client provides $K_a^{\mathsf{C}}{:}(x_{a,\sigma}^{\mathsf{C}})'$ to the Server, which persists $K_a^{\mathsf{C}}{:}(x_{a,\sigma}^{\mathsf{C}})'$ and $K_a^{\mathsf{S}}{:}(x_{a,\sigma}^{\mathsf{S}})'$ as the current $K_a^{\mathsf{C}}{:}x_{a,\sigma}^{\mathsf{C}}$ and $K_a^{\mathsf{S}}{:}x_{a,\sigma}^{\mathsf{S}}$, respectively.

4.6 Signing

Signing uses a selected suite to produce a signature S over an exact application message m. The Client submits a Signing request with class = sign, bound to userAccountId, signingAccountId, suiteId, signPayload, and requestId.

The Server recovers K_a^{S} from wrap_key:K_a^{S} and $x_{a,\sigma}^{\mathsf{S}}$ from $K_a^{\mathsf{S}}{:}x_{a,\sigma}^{\mathsf{S}}$ and responds with $K_u^{\mathsf{C}}{:}K_a^{\mathsf{C}}$ and $K_a^{\mathsf{C}}{:}x_{a,\sigma}^{\mathsf{C}}$; the Client then recovers K_a^{C} from $K_u^{\mathsf{C}}{:}K_a^{\mathsf{C}}$ and $x_{a,\sigma}^{\mathsf{C}}$ from $K_a^{\mathsf{C}}{:}x_{a,\sigma}^{\mathsf{C}}$.

The Client and Server run MPC-TSS interactive threshold signing over m, yielding S, which is verifiable under $pk_{a,\sigma}$.

5 Security Rationale

Continuity targets a setting in which Server-held state alone forms no signing quorum and does not expose plaintext Client-protected material unless the adversary also recovers the user's password.

5.1 Threat Model & Assumptions

The threat model considers three cases: a network attacker observing, replaying, reordering, or substituting messages; a snapshot attacker obtaining persisted Server-held protocol state but neither wrap_key nor the aPAKE Server-side secret material; and the same snapshot attacker, under Server-secret exposure, additionally obtaining wrap_key and the aPAKE Server-side secret material but neither controlling the running Server nor gaining the ability to modify account state.

Across these cases, Continuity assumes the Client and Server erase secret material held in plaintext for an operation or authenticated session when no longer required, including after an operation aborts or fails to commit; the Server persists non-root wrapping keys and signing shares only in their specified wrapped forms; and the Server presents the current state accurately and accepts an operation request only if its identifiers and target protocol state satisfy the requested operation's requirements.

5.2 Analysis

Against a network attacker, the independently authenticated registration channel protects Signup, the aPAKE exchange authenticates the parties during Signin, and the transport instantiated

from transport_key protects subsequent authenticated-session messages.

A snapshot attacker lacks both recovery_key and wrap_key, and the persisted per-registration state alone does not permit offline verification of password guesses. The attacker therefore cannot unwrap either signing share, so the snapshot provides no signing quorum.

Server-secret exposure reveals wrap_key, allowing the adversary to recover $x^{\mathsf{S}}_{a,\sigma}$, and may enable offline verification of password guesses under the selected aPAKE profile's assumptions. Unless the adversary recovers the password and thereby recovery_key, $x^{\mathsf{C}}_{a,\sigma}$ remains protected, so the exposure provides no signing quorum.

5.3 Limitations and Non-Goals

Server availability (including refusal or withholding of artifacts), recovery from password loss, Client compromise, post-compromise security, and rollback protection for persisted protocol state are out of scope.

A Non-Normative Profile

The following profile illustrates one possible selection of the foundational components in Section 3:

aPAKE: OPAQUE-3DH over Ristretto255 and SHA-512 using RFC 9807's recommended configuration; OPAQUE uses the canonical encoding of userAccountId for both `client_identity` and `credential_identifier`, binds each registration record and each authenticated exchange to the intended Server's configured OPAQUE AKE public key, and uses the canonical encoding of the expected protocol, version, and profileId values as `context`; its `export_key` and `session_key` are recovery_key and transport_key, respectively; registration uses a Server-authenticated TLS 1.3 channel [7, 8].

MPC-TSS: 2-of-2 FROST with the Ed25519 ciphersuite from RFC 9591, DKG from the original FROST paper, and DKG-based, public-key-preserving share refresh from `frost-core` 3.0.0 [9, 10, 11].

Canonical encoding: CBOR satisfying RFC 8949's length-first core deterministic encoding requirements, with decoders rejecting encodings that do not satisfy those requirements [12].

AEAD: ChaCha20-Poly1305 with a 256-bit key derived per artifact from its wrapping key using HKDF instantiated with SHA-512/256, a fresh random 256-bit salt, and the canonical encoding of the artifact's class value as `info`; the canonical encoding of the artifact's ctx is used as AAD, and the derived key is used for one encryption with a fresh 96-bit nonce and is not persisted [13, 14, 15].

CSPRNG: The operating system's CSPRNG provides all required randomness.

References

[1] Turnkey Team. *Turnkey's Architecture*. Whitepaper, January 2025.

[2] DFNS. *MPC Signing Infrastructure*. Documentation, last modified June 8, 2026.

[3] P. Wuille. *BIP 32: Hierarchical Deterministic Wallets*. Bitcoin Improvement Proposal, 2012.

[4] M. Palatinus, P. Rusnak, A. Voisine, and S. Bowe. *BIP 39: Mnemonic Code for Generating Deterministic Keys*. Bitcoin Improvement Proposal, 2013.

[5] S. Dutta, P. S. Roy, R. Safavi-Naini, and W. Susilo. *Password Protected Universal Thresholdizer*. ACISP 2026, LNCS 16793, pp. 194–221, Springer Nature Singapore, 2026.

[6] S. Dziembowski, S. Jarecki, P. Kędzior, H. Krawczyk, C. N. Ngo, and J. Xu. *Password-Protected Threshold Signatures*. IACR Cryptology ePrint Archive, Report 2024/1469, 2024.

[7] D. Bourdrez, H. Krawczyk, K. Lewi, and C. A. Wood. *The OPAQUE Augmented Password-Authenticated Key Exchange (aPAKE) Protocol*. RFC 9807, 2025.

[8] E. Rescorla. *The Transport Layer Security (TLS) Protocol Version 1.3*. RFC 9846, 2026.

[9] D. Connolly, C. Komlo, I. Goldberg, and C. A. Wood. *The Flexible Round-Optimized Schnorr Threshold (FROST) Protocol for Two-Round Schnorr Signatures*. RFC 9591, 2024.

[10] C. Komlo and I. Goldberg. *FROST: Flexible Round-Optimized Schnorr Threshold Signatures*. IACR Cryptology ePrint Archive, Report 2020/852, 2020.

[11] Zcash Foundation. *`frost-core` 3.0.0 `refresh_dkg_shares` API Documentation*. `refresh_dkg_shares`, 2026.

[12] C. Bormann and P. Hoffman. *Concise Binary Object Representation (CBOR)*. RFC 8949, 2020.

[13] National Institute of Standards and Technology. *Secure Hash Standard (SHS)*. FIPS PUB 180-4, 2015.

[14] H. Krawczyk and P. Eronen. *HMAC-Based Extract-and-Expand Key Derivation Function (HKDF)*. RFC 5869, 2010.

[15] Y. Nir and A. Langley. *ChaCha20 and Poly1305 for IETF Protocols*. RFC 8439, 2018.